

1. 1 基础知识

1. 1.1 输入与输出

2. 1.2 输入与输出重定向

1. 1.2.1 输出重定向

2. 1.2.2 输入重定向

3. 1.3 各种引号的正确使用

4. 1.4 变量

5. 1.5 基本运算符

2. 2 条件判断与控制

1. 2.1 字符串的判断与比较

2. 2.2 整数判断与比较

3. 2.3 文件属性的判断与比较

4. 2.4 []和[]区别

5. 2.4 条件语句

6. 2.5 case语句

3. 3 循环控制

1. 3.1 for循环

2. 3.2 while循环

3. 3.3 until和select

1. 3.3.1 until

2. 3.3.2 select

4. 3.4 中断与退出

5. 3.5 其他相关

4. 4 数组

1. 4.1 索引数组

1. 4.1.1 定义数组

2. 4.1.2 访问数组

2. 4.2 关联数组

5. 5 Subshell与启动进程

1. 5.1 启动子shell方式

2. 5.2 创建进程

6. 6 函数

1. 6.1 trap信号捕获

2. 参数传递之xargs

3. 6.2 使用shift移动位置参数

7. 7 sed指令

1. 7.1 基本指令

2. 7.2 高级指令

8. 8 awk

1. 8.1 基本语法

2. 8.2 条件判断

3. 8.3 数组与循环

4. 8.4 awk函数

9. 9 shell常用扩展

10. 9.1 花括号扩展

1. 9.2 波浪号

2. 9.3 变量替换

3. 9.4 变量替换

4. 9.5 算术替换

5. 9.6 进程替换

6. 9.7 单词切割

7. 9.8 路径替换

11. 常用命令

1 基础知识

1.1 输入与输出

Linux系统中使用 **echo**和 **printf**实现输出 **echo echo**命令主要用来显示字符串信息。

printf printf [格式] 参数, 基本同C语法一致

格式字符	功能描述
%d或者%i	十进制整数
%o	八进制整数
%x	十六进制整数
%u	无符号十进制整数
%f	浮点数
%s	字符串
\b	退格键
\f	换行但光标仍停留在原来的位置

格式字符	功能描述
\n	换行且光标移至行首
\r	光标移至行首，但不换行
\t	Tab键

举例：

```
printf "%-5d"    #左对齐，打印宽度5
```

printf命令输出信息后，默认是不换行的！

read

选项	功能
-p	显示提示信息
-t	设置读入的超时时间
-n	设置读取n个字符之后结束，而默认会读取标准输入的一整行内容
-r	支持读取\，而默认read命令理解\为特殊字符（转义字符）
-s	静默模式，不显示标准输入的内容

举例：

```
read key1 key2 key3    #读取标准输入到key1 key2 key3，以空格
read -t 3 -p "请输入用户名：" user    #设置超时时间，3s后read 退出
```

1.2 输入与输出重定向

1.2.1 输出重定向

如果希望改变输出信息的方向，可以使用>或>>符号将输出信息重定向到文件中。

使用1>或1>>可以将标准输出信息重定向到文件（1可以忽略不写，默认值就是1），也可以使用2>或2>>将错误的输出信息重定向到文件。这里使用>符号将输出信息重

定向到文件，如果文件不存在，则系统会自动创建该文件，如果文件已经存在，则系统会将该文件的所有内容覆盖（原有数据会丢失！）。

而使用>>符号将输出信息重定向到文件，如果文件不存在，则系统会自动创建该文件，如果文件已经存在，则系统会将输出的信息追加到该文件原有信息的末尾。

举例： 标准输出重定向

```
echo "hello the world" >test.txt  #导出数据到文件
echo "test file" >>test.txt      #追加重定向
```

标准错误重定向 使用2>或者 2>>实现

```
ls -l /nofiles 2> test.txt      #错误重定向
ls -l /oops 2 >>test.txt        #错误重定向， 追加数据
```

同时重定向标准输出和错误输出

```
ls -l /etc/hosts /nofile >ok.txt 2> error.txt  #标准输出和错误输出重定向到不同文件
ls -l /etc/hosts /nofile &> test.txt           #标准输出和错误输出重定向到一个文件
(覆盖方式, &>>是追加方式)
ls -l /nofile 2>&1      #将错误输出重定向到标准输出
ls -l /nofile 1>&2      #将标准输出重定向到错误输出
```

Linux系统中有一个特殊的设备/dev/null，这是一个黑洞。无论往该文件中写入多少数据，都会被系统吞噬、丢弃。

```
echo "hello" >/dev/null
```

1.2.2 输入重定向

可以使用<符号进行输入重定向。<符号后面需要跟一个文件名，这样可以让程序不再从键盘读取输入数据，而从文件中读取数据。

使用<<符号可以将数据内容重定向传递给前面的一个命令，作为命令的输入。

1.3 各种引号的正确使用

引号类型	作用
“” 双引号	作用是引用一个整体
” 单引号	引用一个整体，同时会屏蔽掉特殊符号
`` 反引号	命令替换，使用命令的输出结果替代命令, 注意``不支持嵌套，\$()与反引号作用相同，支持嵌套

举例：

```
touch a b c      #创建三个文件，分别是a、b、c
touch "a b c"    #创建一个文件

echo “###”      #输出空白
echo ‘###’       #输出#号

test=18
echo "$test RMB"  #输出18 RMB
echo '$test RMB'  #输出$test RMB

tar -czf /root/log-`date +%Y%m%d`.tar.gz /var/log/
```

1.4 变量

定义变量时等号两边不可以有空格。

当需要读取变量值时，需要在变量名前添加一个美元符号“\$”；而当变量名与其他非变量名的字符混在一起时，需要使用{}分隔。

举例：

```
test=123
echo $test      #打印123
echo $testRMB   #空
echo ${test}RMB #打印123RMB
```

系统预设变量

变量名	描述
UID	当前账户ID号
USER	当前账户的名称
HISTSIZE	当前终端的醉酒历史命令条目数量
HOME	当前账户的根目录
LANG	当前环境使用的语言
PATH	命令搜索路径
PWD	返回当前的工作目录
RANDOM	随机返回0-32767的整数
\$0	当前的命令的名称
\$n	返回位置参数，如1表示第一个参数，数字大于9时需要使用{n}
\$#	命令参数的个数
\$*	命令行的所有参数，所以参数作为一个整体
\$@	命令行的所有参数，所有参数作为独立的个体
\$?	返回上一条命令退出时的状态代码
\$\$	返回当前进程的进程号
#!	返回最后一个后台进程的进程号

1.5 基本运算符

可以使用((表达式))、[表达式]、let表达式进行整数的算数运算 举例：

```
echo $((2+4))
echo $((2**4)) #2的4次幂

echo ${2**8}
```

使用let命令计算时，默认不会输出运算的结果，一般需要将运算的结果赋值给变量，通过变量查看运算结果。另外，使用let命令对变量进行计算时，不需要在变量名前添加\$符号。

```
let 1+2 ##无输出
x=5
let x++
echo $x ##输出6
```

```
${#var} : 计算出变量的长度
如:
a=cxndicd
echo ${#a} #结果显示7
```

2 条件判断与控制

在Shell中可以使用多种方式进行条件判断，如[[表达式]]、[表达式]或者test表达式。

需要注意的是，不管使用哪种方式进行条件判断，系统默认都不会有任何输出结果，可以通过echo \$? 命令，查看上一条命令的退出状态码，或者使用&&和||操作符结合其他命令进行结果的输出操作。

表达式两边必须有空格，否则程序会出错。使用[[]]和test进行排序比较时，使用的比较符号不同。在test或[]中不能直接使用<或>符号进行排序比较。

如果需要在一行代码中输入多条命令，在Shell中可以使用：（分号）、&&（与）、||（或）这三个符号将多个命令分隔。

- ；（分号）是按顺序执行命令，分号前后的命令可以没有任何逻辑关系。例如，输入“A命令；B命令”，系统会先执行A命令，不管A命令执行结果如何，都会执行B命令。整个命令的退出码以最后一条命令为准，B命令如果执行成功则退出码为0，B命令如果执行失败则退出码为非0。
- &&（与）符号分隔多条命令时，仅当前一条命令执行成功后，才会执行&&后面的命令。例如，输入“A命令&&B命令”，系统会先执行A命令，如果A命令执行成功则执行B命令，如果A命令执行失败则不执行B命令。而整行命令的退出码取决于两条命令是否同时执行成功，如果A命令执行成功并且B命令执行也成功，则整行命令的退出码为0，而A命令或B命令中的任何一条命令执行失败，则整行命令的退出码为非0。
- ||（或）符号分隔多条命令，仅当前一条命令不执行或执行失败后才执行后一条命令。例如，输入“A命令||B命令”，因为A命令是命令行的第一条命令，所以一定会执行，如果A命令执行成功了就不再执行B命令，如果A命令执行失败，则执行B命令，A命令和B命令为二选一的关系。A命令或B命令中有任何一条命令的退出码为0，则整行命令的退出码就是0，否则返回非0。

2.1 字符串的判断与比较

使用test或[]测试的效果是一样，表达式中可以使用变量。

在Shell中进行条件测试时一定要注意空格问题。使用[]测试时，左方括号右边和右方括号左边都必须有空格。而且测试的比较符号两边也必须都有空格。

```
test a == a #测试字符串是否相等

##下面两者等价
test $USER == root
[ $USER == root ]
```

参数

参 数	说明
-z	测试一个字符串是否为空
-n	测试一个字符串是否非空， 最好用双引号将字符串包含，因为如果变量为空，会被认为有值，因为空格也是值。

举例：

```
[ -z $test ] && echo Y || echo N #输出Y

test=123
[ -z $test ] && echo Y || echo N #输出N
```

2.2 整数判断与比较

比较符号	含义
-eq	等于
-ne	不等于
-gt	大于

比较符号	含义
-ge	大于或等于
-lt	小于
-le	小于或等于

举例：

```
test 3 -eq 3 && echo Y || echo N #打印Y
[ 6 -gt 4 ] && echo Y || echo N #打印Y
```

2.3 文件属性的判断与比较

操作符	功能描述
-e file	判断文件或目录是否存在，存在返回真，否则返回假
-f file	判断存在且为普通文件
-d file	判断存在且为目录
-b file	判断存在且为块设备文件（如磁盘）
-c file	判断存在且为字符设备文件（如键盘）
-L file	判断存在且为软连接文件
-p file	判断存在且为命名管道
-r file	判断存在且当前用户对该文件具有可读权限
-w file	判断存在且当前用户对该文件具有可写权限
-x file	判断存在且当前用户对该文件具有可执行权限
-s file	判断存在且文件大小非空
file1 -ef file2	两个文件使用相同设备、相同inode号，则返回真，否则返回假
file1 -nt file2	file1比file2更新时返回真，或者file1存在而file2不存在时返回真
file1 -ot file2	file1比file2更旧时返回真，或者file2存在而file1不存在时返回真

举例：

```
touch ver1.txt
touch ver2.txt
```

```
[ -e ver1.txt ] && echo Y || echo N    ##打印Y
```

注意：在测试权限时需要注意，超级管理员root在没有rw权限的情况下，也是可以读写文件的，rw权限对超级管理员是无效的。但是如果文件没有x权限，哪怕是root也不可以执行该文件。在使用-r和-w判断时需要注意

2.4 [[]]和[]区别

多数情况下[]和 [[]]是可以通用的，两者的主要差异是：test或[]是符合POSIX标准的测试语句，兼容性更强，几乎可以运行在所有Shell解释器中，相比较而言 [[]]仅可运行在特定的几个Shell解释器中（如Bash、Zsh等）。

区别：（1）在 [[]]中使用<和>符号时，系统进行的是排序操作，而且支持在测试表达式内使用&&和||符号。在test或[]测试语句中不可以使用&&和||符号。

（2）[]也支持同时进行多个条件的逻辑测试，但是在[]中需要使用-a和-o进行逻辑与和逻辑或的比较操作，而 [[]]中可以直接使用&&和||进行逻辑比较操作，更直观，可读性更好。

- A && B或者A -a B，意思是仅当A和B两个条件测试都成功时，整体测试结果才为真。
- A || B或者A -o B，意思是只要A或B中的任意一个条件测试成功，则整体测试结果为真。

（3）需要注意的还有==比较符，在 [[]]中==是模式匹配，模式匹配允许使用通配符。例如，Bash常用的通配符有*、?、[...]等。而==在test语句中仅代表字符串的精确比较，判断字符串是否一模一样。

（4）在 [[]]中还支持使用=~进行正则匹配，而在[]中则完全不支持正则匹配。（5）分组测试，使用（）进行分组，效果类似于虽然在数学上默认先算乘除法再算加减法，但使用（）后可以先算加减法再算乘除法。

```
[[ $name =~ [a-z] ]] && echo Y || echo N    #判断name的值是否包含小写字母
```

[[]]测试	[]测试
< 排序比较	不支持，仅部分 shell 支持<
> 排序比较	不支持，仅部分 shell 支持>
&& 逻辑与	-a 逻辑与
== 模式匹配	== 字符匹配
=~ 正则匹配	不支持
() 分组测试	()仅部分 shell 支持分组测试

2.4 条件语句

if语句后面的条件测试语句不一定非要是**test**或[]测试语句，任何有返回值的命令都可以写在if语句后面，命令返回值为0代表执行成功（即为真），返回值非0代表执行失败（即为假）。

语法格式：

```
#单分支：
if 条件测试
then
    命令序列
fi

或者
if 条件测试; then
    命令序列
fi

#双分支：
if 条件测试
then
    命令序列1
else
    命令序列2
fi
或者
if 条件测试; then
    命令序列1
else
    命令序列2
fi

#多分支
if 条件测试1; then
    命令序列1
```

```
elif 条件测试2; then
    命令序列2
... ..
else
    命令序列n
fi
```

举例：

```
#!/bin/bash
clear
num=$((RANDOM%10+1))
read -p "请输入1~10之间的整数: " guess

if [ $guess -eq $num ];then
    echo "恭喜猜对了, 就是: $num"
elif [ $guess -lt $num ]; then
    echo "Oops, 猜小了"
else
    echo "Oops, 猜大了"
fi
```

2.5 case语句

语法：

```
case word in

    模式1)
        命令序列1; ;
    模式2)
        命令序列2;;
    .....
    *)
        命令序列n; ;
esac

##多模式匹配，case命令可以使用管道符号（|）进行多个模式的匹配
case word in
    模式1|模式2|模式3)
        命令序列1; ;
    模式4|模式5|模式6)
        命令序列2;;
    ... ..
    *)
        命令序列n;;
esac
```

如果命令序列的最后使用了；；（双分号），则**case**命令不再对后续的模式进行匹配比较，即匹配停止。如果使用；&替代；；会导致**case**继续执行下一个模式匹配中附加的命令序列。如果使用；；&替代；；则会导致**case**继续对下一个模式进行匹配，如果匹配则执行对应命令序列中的命令。

```
read -p "请输入一个a~c之间的字母: " key
case $key in
a)
    ##使用;;&会继续对后面的模式进行匹配
    ##所以屏幕会继续显示后面的I am aa
    echo "I am a.";&
b)
    echo "I am b.";;
a)
    ##使用;&会执行后一个模式匹配中的命令
    ##所以屏幕会继续显示I am c.
    echo "I am aa.";&
c)
    echo "I am c.";;
a)
    echo "I am aaa.";;
*)
    echo "Out of rang";;
esac
```

3 循环控制

3.1 for循环

(1) 基本语法：

```
for name [ in [ word ... ] ]
do
    命令序列
done
```

在该基本语法格式中，**name**是可以任意定义的变量名称，**word**是支持扩展的项目列表，扩展后生成一份完整的项目列表（或值列表）。**name**会逐一提取项目列表中的每一个值，每提取一个值就会执行一次**do**和**done**中间的命令序列。

举例：

```
for i in 1 2 3 4 5
do
    echo "$i hello world";
done
```

(2) 不定义取值范围

```
for name
do
    命令序列
done
```

如果变量`name`没有定义取值范围，则默认取值为`$@`，也就是所有位置变量的值。这样有几个位置变量，该`for`循环语句就循环几次。

举例：

```
## 脚本名称 demo.sh
#!/bin/bash
for i
do
    echo $i
done

./demo.sh hello 798 beijing
打印结果是：
hello
798
beijing
```

`for`循环支持多种扩展，如变量替换、命令扩展、算术扩展、通配符扩展等

举例：

```
for i in {1..5000}  ##在1到5000之间
for i in $(seq 254) ##在1到254之间
```

(3) c风格for循环

```
for ((expr1; expr2; expr3))
do
```

命令序列

done

举例

```
for ((i=1;i<=5;i++))
do
    echo $i
done
```

3.2 while循环

while循环语法：

```
while 条件判断
do
    命令序列
done
```

举例

```
i = 1
while [ $i -le 5 ]
do
    echo "hello world"
    let i++          #此处需要计算i表达式，因此不能直接使用i++
done
```

while命令后面的条件判断只要语句命令返回码为**0**就代表真，否则代表假。并非仅仅可以写[]或[]判断，**while**的判断可以是任何可以执行的命令。

小技巧 - 死循环

```
(1) while true
(2) while :
```

小技巧 - read

(1)重定向输入

```
while IFS=":" read user pass uid gid info home shell #临时修改IFS为:
do
    echo -e "My UID:$uid"
done < /etc/passwd #重定向输入为passwd文件
```

(2)管道输入

```
df -h | grep ^/ | while read name size other
do
    echo $name $size
done
```

3.3 until和select

3.3.1 until

基本语法

```
until 条件判断
do
    命令序列
done
```

与while语句相反，until循环语句只有当条件判断结果为真时才退出循环，而当条件判断结果为假时则执行循环体中的命令。

实例

```
i=1
until [ $i -ge 5 ]
do
    echo $i
    let i++
done
```

3.3.2 select

基本语法

```
select 变量名 in 值列表
do
```

实例

```
echo "请根据提示选择一个选项"

select item in "CPU" "IP" "MEM" "exit"
do
    case $item in
        "CPU")
            uptime;;
        "IP")
            ip a s;;
        "MEM")
            free;;
        "exit")
            exit;;
        *)
            echo error;;
    esac
done
```

3.4 中断与退出

continue

continue命令可以结束单次循环，**continue**命令后面的所有语句不再执行，进而直接跳转到下一次循环。如果脚本使用了循环的嵌套功能，则**continue**命令后面可以跟数字参数（数字要求大于或等于1），表示对第几层循环执行跳出操作(比如当前循环认为是1，2表示从当前循环向外数在数一层)。但是，注意：这里的跳出操作并不是让整个循环结束，而是仅仅结束这一次循环，并进入下一次循环。

break

中断命令**break**，该命令可以结束整个循环体，**break**后面的所有语句不再执行，并且整个循环提前结束。如果脚本使用了循环的嵌套功能，则**break**命令后面可以跟数字参数（数字要求大于或等于1），表示对第几层循环执行中断。

exit

中断级别最高的命令**exit**，该命令会直接结束整个脚本，**exit**后面也可以跟数字参数，表示脚本的退出状态，如果没有指定数字参数，则脚本的退出状态就是上一个命令的退出状态。

3.5 其他相关

IFS: 在Shell中使用内部变量IFS（Internal Field Separator）来决定项目列表或值列表的分隔符，IFS的默认值为空格、Tab制表符或换行符，可以通过环境变量 `$IFS` 查看。在Shell中使用内部变量IFS（Internal Field Separator）来决定项目列表或值列表的分隔符，IFS的默认值为空格、Tab制表符或换行符。

for循环的列表分割、或者read读取多个输入都是以 `$IFS` 作为分割：

```
(1) read -p "请输入3个数据" x y z #默认以空格等分割
(2) X = "a b c d"
for i in $X
do
    echo "I am $i" #以空格等分割
done
```

可以自定义设置IFS的值，比如 `IFS=":"`，但是如果要设置特殊的控制字符作为分隔符，必须使用 `$'string'` 方式：

```
IFS=$'\t' #设置Tab制表符
IFS=$' \t\n' # 设置空格、制表符和换行符
```

4 数组

4.1 索引数组

4.1.1 定义数组

语法：

```
#方式一
数组名[索引 1]=值1
数组名[索引 2]=值2
数组名[索引 n]=值n
#方式二，系统默认使用以0开始的有序数字作为索引，使用空格符分隔
数组名=(值1 值2 值3 ...)
```

举例：

方式一

```
name[0]="Jacob"
name[1]="Rose"
name[4+4]="TinTin"
```

方式二

```
name1=(Jacob Rose TinTin)
```

#该方式也可以使用\$() 或者``将命令的执行结果赋值给数组

```
root=$(df / | tail -n + 2) # df / | tail -n + 2表示删除标题，从第二行开始显示
```

4.1.2 访问数组

索引可以是算术表达式，但要求运算的结果是整数。可以通过索引定义数组，同样也可以使用索引获取数组中某个元素的值。注意，数字索引可以是一个变量，索引可以不连续。

```
echo ${name[0]} # 数组访问认为是表达式，因此使用${}括起来
echo ${name[2]} #访问不存在的索引，返回空
echo ${name[8]}
echo ${name[4+4]} #索引可以使用算术表达式
echo ${name[-1]} #数组中最后一个元素
echo ${name[-2]} #数组中倒数第二个元素
echo ${name[*]} #查看数组中所有元素， 所有元素被视为一个整体
echo ${name[@]} #列出数组中所有元素，所有元素被视为独立个体
echo ${name[$i]} #i=1，数组索引可以使变量
```

```
echo ${#name[*]} #统计数组中元素个数，此处输出3， 原理：${#var}统计var长度，name[*]是
“Jacob Rose TinTin”，其长度为3（空格分隔）
```

```
name[8]="Tin" #修改数组元素
name[$i]="Tim" #i=8
```

4.2 关联数组

语法格式：

方式一：

```
declare -A 数组名
数组名[key1]=值1
数组名[key2]=值2
```

方式二：

```
declare -A 数组名  
数组名=([key1]=值1 [key2]=值2 ...)
```

实例：

```
declare -A man  
man[name]=Tom  
man[age]=26  
  
declare -A woman  
woman=([name]=lucy [age]=25)  
  
unset woman[age]  #删除数组中某个元素  
unset woman      #删除数组变量
```

5 Subshell与启动进程

通过当前Shell启动的一个新的子进程或子Shell被称为SubShell（子Shell）。子Shell会自动继承父Shell的很多环境，如变量、工作目录、文件描述符等，但是反之，子Shell中的环境仅在子Shell中有效，父Shell无法读取子Shell的环境。

通过Shell变量BASH_SUBSHELL可以查看子Shell的信息，该变量的初始值为0，每启动一个子Shell该变量的值会自动加1。

5.1 启动子shell方式

方式一：使用(), 也可以生成多级子shell

```
#!/bin/bash  
hi="hello, 我是父shell"  
echo "bash_subshell=$BASH_SUBSHELL"  
#通过()开启子shell  
(  
  sub_hi="hello, 我是子shell"  
  echo -e "bash_subshell=$BASH_SUBSHELL"  
)
```

方式二: 使用管道开启子shell

5.2 创建进程

fork方式

通常情况下在系统中通过相对路径或绝对路径执行一个命令时，都会由父进程开启一个子进程，当子进程结束后再返回父进程，这种行为过程就叫作**fork**。当脚本中正常调用一个外部命令或其他脚本时，都会**fork**一个子**Shell**进程，我们的命令会运行在这个子**Shell**中。 实例：

```
#!/bin/bash
sleep 5      #调用一个外部命令会fork一个子进程
/root/tmp.sh #相对路径或绝对路径执行一个命令时会fork一个子进程
```

exec方式

使用**exec**方式调用其他命令或脚本时，系统不会开启子进程，而是使用新的程序替换当前的**Shell**环境，因为当前**Shell**环境被替换了，所以当**exec**调用的程序结束后，当前环境会被关闭。

```
exec ls #使用exec方式调用其他命令或脚本时，系统不会开启子进程，而是使用新的程序替换当前的Shell环境，因为当前Shell环境被替换了，所以当exec调用的程序结束后，当前环境会被关闭.为了防止当前脚本被覆盖，一般都会将exec写入另一个脚本，先使用fork方式调用该脚本，然后在fork的子进程中调用exec命令
```

```
echo "test"
cd /etc
```

source方式

使用**source**命令或.（点）可以不开启子**Shell**，而在当前**Shell**环境中将需要执行的命令加载进来，执行完加载的命令后，继续执行脚本中后续的指令。

6 函数

定义函数的方式：

```
方式一：
函数名称() {
    代码序列
}
```

```
方式二：
function 函数名() {
    代码序列
}
```

```
方式三：
function 函数名 {
    代码序列
}
```

注意：通过函数名调用函数时，无需像C++一样使用()

```
mymkdir() {
    mkdir /tmp/test
    touch /tmp/test/hi.txt
}

mymkdir  ##调用函数

unset mymkdir  ##取消函数定义
```

在函数体内部可以通过变量\$1、\$2读取位置参数，在调用函数时添加相应的参数即可，或者读取其他全局变量都可以实现传递变量参数的功能(无需在函数定义的小括号内定义形参)。

函数体内 \$@可以读取所有位置参数。

6.1 trap信号捕获

kill -l可以显示所有信号 trap '命令' 信号列表 可以在shell中捕获信号，但不能捕获所有信号，像TERM、KILL之类的信号是无法被捕获的。 例子：

```
trap 'echo "打死不中断";sleep 3' INT TSTP
trap 'echo "测试"; sleep 3' HUP

while:
do
    echo "signal"
    echo "demo"
done
```

参数传递之xargs

对于不支持从管道读取输入的程序，比如find、cut、rm、kill等可以通过xargs解决。比如：

```
find /etc/ -name *.conf -type f | echo    # 输出结果为空
echo /tmp/test.txt | rm                  # 执行命令报错
```

xargs命令就可以读取标准输入或管道中的数据，并将这些数据传递给其他程序作为参数。不指定程序时xargs默认会将数据传递给echo命令。

```
find /etc/ -name *.conf -type f | xargs echo    # 显示查找的文件名
echo /tmp/test.txt | xargs rm                  # 删除测试文件
```

默认xargs读取参数时以空格、Tab制表符或者回车符为分隔符和结束符，但是有些文件名本身可能就包含有空格，此时xargs会理解一个文件有多个参数。

文件名本身可能就包含有空格，此时xargs会理解一个文件有多个参数。

```
touch "hello world.txt"

find ./ -name "*.txt" | xargs rm
#输出是：
rm: cannot remove './hello': No such file or directory
rm: cannot remove 'world.txt': No such file or directory
```

相关选项

```
xargs -a /etc/redhat-release echo    # -a从文件中读取参数
seq 5 | xargs -n 2                   # -n指定一次读取几个参数
echo "helloattheword" | xargs -da   # -d指定任意字符为分隔符
```

6.2 使用shift移动位置参数

shift命令后面需要一个非负整数作为参数，如果没有指定该参数，则默认为1。shift命令后面需要一个非负整数作为参数，如果没有指定该参数，则默认为1。左移之后，右边空出的参数将变为空，参数个数减少

```
# 脚本参数为shift.sh
echo "arg1=$1, arg2=$2, arg3=$3, count=$#"
```

```
shift
echo "arg1=$1, arg2=$2, arg3=$3, count=$#"

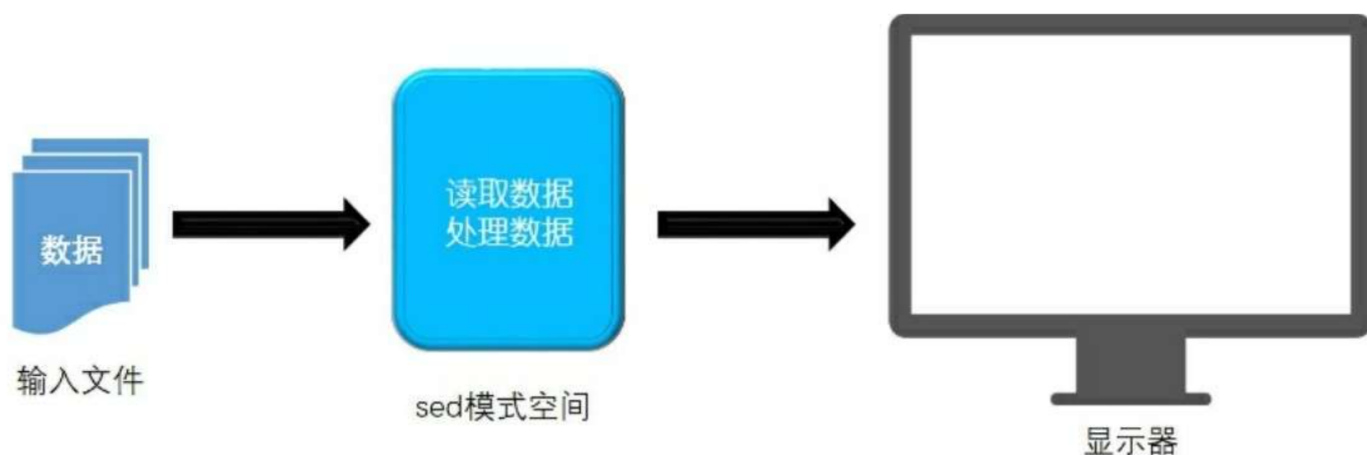
执行shift.sh a b c
# 输出
arg1=a, arg2=b, arg3=c, count=3
arg1=b, arg3=c, arg3=, count=2
```

7 sed指令

sed是行处理编辑器。

sed会逐行扫描输入的数据，并将读取的数据内容复制到缓冲区中，我们称之为模式空间，然后拿模式空间中的数据与给定的条件进行匹配，如果匹配成功则执行特定的sed指令，否则sed会跳过输入的数据行，继续读取后续的数据。

sed是逐行处理软件，我们可能仅输入了一条sed指令，但系统会将该指令应用在所有匹配的数据行上，因此相同的指令会被反复执行N次，这取决于匹配到的数据有几行。



7.1 基本指令

语法格式：

```
命令 | sed [选项] '匹配条件和操作指令'
sed [选项] '匹配条件和操作指令' 输入文件...
```

命令选项

选项	功能描述
-n, --silent	屏蔽默认输出功能，默认sed把匹配到的数据显示在屏幕上

选项	功能描述
-r	支持扩展正则
-i[SUFFIX]	直接修改源文件，如果设置了SUFFIX后缀名，sed会将数据备份
-e	指定需要执行的sed命令，支持使用多个-e参数
-f	指定需要执行的脚本文件，需要提前将sed指令写入文件

基本操作指令

操作指令	功能描述
p	打印当前匹配的数据行
l	小写l，打印当前匹配的数据行，会显示控制字符，比如回车符等
=	打印当前读取的数据行数
a text	在匹配数据行后面追加文本内容
i text	在匹配数据行前面插入文本内容
d	删除匹配的数据行整行内容
c text	将匹配的数据行整行内容替换为特定的文本内容
r filename	从文件中读取数据并追加到匹配的数据行后面
w filename	将当前匹配到的数据写入特定的文件中
q[exit code]	立刻退出sed脚本
s/regexp/replace/	使用正则匹配，将匹配到的数据替换为特定的内容

sed指令执行前需要先根据条件定位需要处理的数据行，如果没有指定定位条件，则默认sed会对所有数据行执行特定的指令。

数据定位方式

格式	功能描述
number	直接根据行号匹配数据
first~step	从first行开始，步长为step，匹配所有满足条件的数据行
\$	匹配最后一行
/regexp/	使用正则表达式匹配数据行

格式	功能描述
<code>\cregexpc</code>	使用正则表达式匹配数据行， <code>c</code> 可以是任意字符
<code>addr1, addr2</code>	直接使用行号定位，匹配从 <code>addr1</code> 到 <code>addr2</code> 的所有行
<code>addr1,+N</code>	直接使用行号定位，匹配从 <code>addr1</code> 开始及后面的 <code>N</code> 行
<code>!</code>	对匹配的条件取反

例子：

```
sed -n 'p' /etc/hosts      #所有行显示一次
sed -n '2p' /etc/hosts    #仅显示文件第二行， 直接根据行号匹配数据
df -h | sed -n '2p'       #支持从管道读取数据
sed -n '1,3p' /tmp/passwd #显示文件的第1行到第3行
sed -n '1p;3p;6p' /tmp/passwd #多条指令使用行号分隔，打印1、3、6行
sed -n '4,$p' /tmp/passwd #打印第4行到末尾所有行
sed -n '3,+3p' /tmp/passwd #打印第3行以及后面的3行
sed -n '1~2p' /tmp/passwd #打印1、3、5行，步长为2
sed -n '$p' /tmp/passwd   #打印文件最后一行的内容
sed -n '/root/p' /tmp/passwd #匹配包含root 的行并打印， /regexp/ 使用正则表达式匹配数据行
sed -n '/^http/p' /tmp/passwd #显示以http开头的数据行

sed -n '/root=' /tmp/passwd #显示包含root的行号
sed -n '3=' /tmp/passwd     #显示第三行的行号
sed -n '$=' /tmp/passwd     #显示最后一行的行号

sed -n '1!p' /etc/hosts     #显示除第1行外的所有行数据

sed '1a add test line' /tmp/host #在第一行后追加 add test line， sed仅仅是在缓存区
中修改了数据，使用-i选项可以修改源文件
sed '1i add new line' /tmp/host  #在第一行前面添加 add new line

sed '1d' /tmp/host          #删除第一行
sed '2c modify line' /tmp/host #将第二行整行替换为 modify line
sed '1r /etc/hostname' /tmp/hosts #read读取其他文件内容追加到/tmp/hosts第一行后面
sed '1,3w /tmp/myhosts' /etc/shells #将1到3行保存为/tmp/myhosts，旧的/tmp/myhosts
将被覆盖
sed '3q' /etc/shells        #读取文件第三行时退出

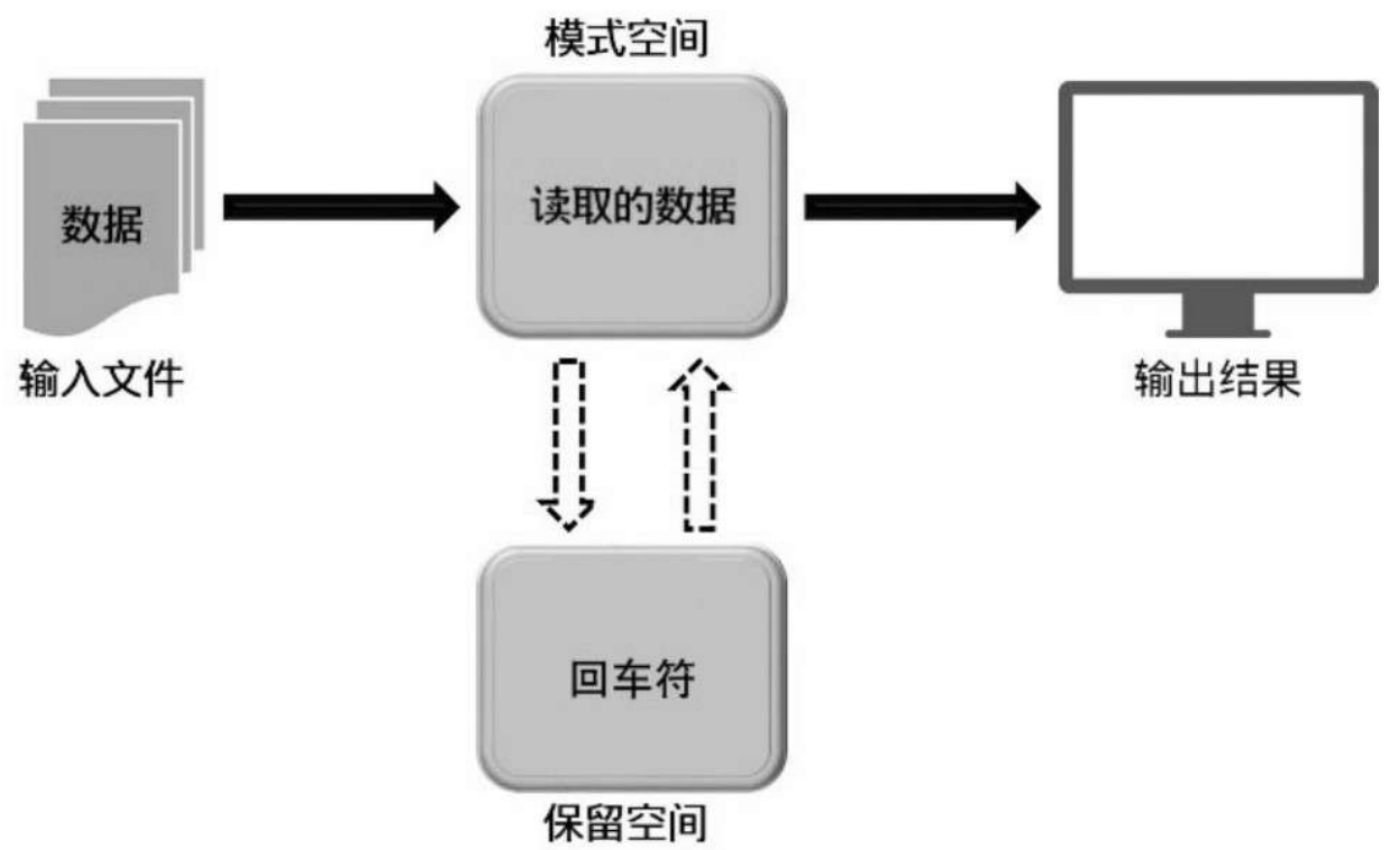
sed 's/hello/hi/' test.txt  #将test.txt中匹配到的hello替换为hi，而不是替换整行
sed '1s/hello/hi/' test.txt #仅仅对第一行进行替换操作
sed 's/o/0' test.txt        #仅将每行的第一个o替换
sed 's/o/0/g' test.txt      #对每行的所有o替换
sed 's/o/0/2' test.txt      #对每行的第二个o替换

sed -f script.sed test.txt  #从script.sed中读取指令执行
```

7.2 高级指令

高级操作指令	功能描述
h	将模式空间中的数据复制到保留空间
H	将模式空间中的数据追加到保留空间
g	将保留空间中的数据复制到模式空间
G	将保留空间中的数据追加到模式空间
x	将模式空间和保留空间中的数据对调
n	读取下一行数据到模式空间
N	读取下一行数据追加到模式空间
y/源/目标	以字符为单位将源字符转为目的字符
:label	为t或b指令定义label标签
t label	有条件跳转到标签，如果没有标签则跳转到指令的结尾
b label	跳转到标签，如果没有标签则跳转到指令的结尾

sed在对数据进行编辑修改前需要先将读取的数据写入模式空间中，而sed除了有一个用于临时存储数据的模式空间，还设计有一个保留空间，保留空间中默认仅包含有一个回车符。前面我们学习的a、i、d、c、s等指令都仅用到了模式空间，而不会调用保留空间中的数据，仅当我们使用特定的指令时（如h、g、x等）才会用到保留空间中的数据，注意在保留空间中默认包含一个回车符。



举例：

文本内容

vim test.txt

1: hello the world.

2: go spurs go.

3: 123 456 789

4: hello the beijing

5: I am Jacob

6: Test hold/pattern space

##example 1

命令: sed '2h;5g' test.txt

回显:

1: hello the world.

2: go spurs go.

3: 123 456 789.

4: hello the beijing.

2: go spurs go.

6: Test hold/pattern space.

说明：读取文件的第2行时将整行数据复制到了保留空间，并将保留空间中原有的回车符覆盖了，然后在读取第5行数据时，使用保留空间中的数据覆盖掉模式空间中的数据

##example 2

命令: sed '2H;5G' test.txt

回显:

1: hello the world.

2: go spurs go.

3: 123 456 789.

4: hello the beijing.

5: I am Jacob

@@

2: go spurs go.

6: Test hold/pattern space.

说明:

将第2行的数据追加到保留空间，保留空间中的回车符不会被覆盖，而到了读取第5行数据时，又使用大写字母G将保留空间中的数据追加到模式空间

##example 3

命令: sed '2h;2d;5g' test.txt

回显:

1: hello the world.

3: 123 456 789.

4: hello the beijing.

2: go spurs go.

6: Test hold/pattern space.

说明:

读取第2行数据时，先将数据复制到了保留空间，复制完成后，删除了第2行的数据，因此屏幕上不再显

示文件原有的第2行数据 (go spurs go)。但是这些数据并没有彻底消失，因为已经被复制到了保留空间，最后当sed读取到文件的第5行时，使用保留空间中的数据将第5行的内容覆盖掉

##example 4

命令: sed '1h;4x' test.txt

回显:

1: hello the world.

2: go spurs go.

3: 123 456 789

1: hello the world.

5: I am Jacob

6: Test hold/pattern space

说明:

读取文件的第一行内容到模式空间，将该行数据复制到保留空间，当读取文件第4行时，执行x指令将模式空间与保留空间的数据互换，模式空间就变成了hello the world

##example 5

命令: sed 'n;d' test.txt

回显:

1: hello the world.

3: 123 456 789

5: I am Jacob

说明:

先是读取test.txt的第一行到模式空间，然后执行n会导致打印当前行的数据内容（显示第一行hello the world），接着读取文件下一行数据到模式空间（go spurs go），此时再执行d就会将刚刚读取进来的数据删除（删除go spursgo这行），最后当n和d指令都执行完毕了，接着sed继续读取文件下一行数据（读取第3行进入模式空间），依此类推。

定义标签需要使用冒号 (:) 开始，后面跟任意标签字符串（标签名称），冒号与标签字符串之间不能有空格，而如果字符串最后有空格，则空格也被理解为标签名称的一部分。

跳转只影响sed指令的执行顺序，对输入的数据行没有影响。

我们可以通过b或者t指令跳转至标签的位置，如果b或者t指令跳转的目标标签不存在，则sed直接跳转至命令结束位置。区别是b为无条件跳转，t为有条件跳转。t需要根据前面的s替换指令的结果决定是否跳转。

无条件跳转

:label

sed指令序列

... ..

b label

有条件跳转

:label

sed指令序列

```
... ..  
s/regex/replace/  
t label
```

```
sed -n ':top;=;p;4b top' text.txt
```

test是一种有条件的跳转，使用时必须和**s**替换操作配合使用。当**s**替换操作成功时则执行**test**跳转，如果跳转的目标标签不存在，则跳转到指令的结束位置，反之，如果**s**替换操作不成功，则不执行**test**跳转操作。

```
sed -r ':start;/:$/N;s/\n+//;t start' contact.txt
```

说明：

在开始执行任何sed指令前先定义了一个名称为start的标签，然后条件匹配以冒号(:)结尾的行，找到满足条件的行后执行N指令读取下一行数据到模式空间，接着使用s替换指令将\n（换行符）以及后面的若干空格都替换为空（即删除操作），最后test有条件跳转，如果前面的s替换操作成功则执行t跳转指令，否则不执行t跳转指令。

8 awk

awk是专门为文本处理设计的编程语言，**awk**与**sed**类似都是以数据驱动的行处理软件，通常我们会使用它进行数据扫描、过滤、统计汇总工作，数据可以来自标准输入、管道或者文件。

8.1 基本语法

```
awk [选项] '条件{动作} 条件{动作} ... ..' 文件名 ...
```

awk是一种处理文本文件的编程语言，文件的每行数据被称为记录，默认以空格或制表符为分隔符，每条记录会被分成若干字段（列），**awk**每次从文件中读取一条记录。

awk语法由一系列条件和动作组成，在花括号内可以有多个动作，在多个动作之间使用分号分隔，在多个条件和动作之间可以有若干空格，也可以没有。

条件可以是正则匹配、数字或字符串比较，动作可以是打印需要过滤的数据或者其他，如果没有指定条件则可以匹配所有数据行，如果没有指定动作则默认为**print**打印操作。

awk内置变量

变量名	描述
FILENAME	当前输入文档的名称
FNR	当前输入文档的当前行号，尤其在有多个输入文档时有用，将多个文档视为独立的
NR	输入数据流的当前行号，将多个文档作为一个整体
\$0	当前行的全部数据内容
\$n	当前行的第n个字段的内容, n>=1
NF	当前记录的字段个数
FS	字段分隔符，默认是空格或者Tab
OFS	输出字段分隔符，默认是空格
ORS	输出记录分隔符，默认是换行符
RS	输入记录分隔符，默认是换行符

举例：

```
free | awk '{print $2}'      #逐行打印第二列
free | awk '{print NR}'     #输出每行行号
free | awk '{print NF}'     #输出每行列数
awk '{print $(NF-2)}' test.txt #输出每行倒数第三列
awk '{print FILENAME}' test.txt #test.txt有几行就输出几个文件名
```

自定义变量 **wk** 可以通过 **-v**（**variable**）选项设置或者修改变量的值，我们可以使用 **-v** 定义新的变量，也可以使用该选项修改内置变量的值。

```
##例1
awk -v x="Jacob" '{print x}' test.txt      #定义变量，并输出变量值

##例2
x="hello" #自定义系统变量
awk -v i=$x '{print i}' test.txt
awk '{print "'$i'"}' test.txt      #双引号+单引号的组合

##例3
awk -v FS=":" '{print $2}' test1.txt      #重新定义分隔符
awk -v FS="[:,-]" '{print $2}' test1.txt  #重新定义冒号、逗号和横线作为分隔符，横线不表示范围时，应该放在开头或者末尾

##例4
awk -F: '{print $1}' test.txt      #使用-F选项自定义分隔符
awk -F"[:,-]" '{print $1}' test.txt
```

##例5

awk -v RS="," '{print \$1}' test1.txt #定义逗号为行分隔符

##例6

awk -v OFS=":" '{print \$1,\$2,\$3}' test1.txt #定义输出字段分隔符，打印的字段间使用：分割

awk -v ORS=":" '{print}' test1.txt #定义输出行分隔符，打印的行间使用：分割

条件匹配

比较符号	描述
//	全行数据正则匹配
!//	对全行数据正则匹配后取反
~//	对特定数据正则匹配
!~//	对特定数据正则匹配后取反
==	等于
!=	不等于
>	大于
>=	大于等于
<	小于
<=	小于等于
&&	逻辑与

awk '/world/{print}' test.txt #打印包含world的行

默认正则匹配是对每行所有数据内容进行匹配，但是**awk**支持仅对某列进行正则匹配

awk '\$2~/the/' test1.txt #对第二列正则匹配the

##该命令仅对每行的第2列进行正则匹配包含the的数据行，第1行的第2列包含the则打印输出该行的所有内容，不包含the则通过跳过该行数据，没有任何数据输出。

awk '\$3~/never/{print \$1,\$2,\$5}' test.txt #逐行匹配第3列是否包含never关键词，如果包含则打印输出该行的第1列、第4列及第5列

awk '\$2!="the"' test.txt #打印第二列不等于the的行

awk的匹配条件可以是**BEGIN**或**END**（大写字母），**BEGIN**会导致动作指令仅在读取任何数据记录之前执行一次，**END**会导致动作指令仅在读取完所有数据记录后执行一次。

```
awk -F: 'BEGIN{print "用户名 UID 解释器"} {print $1,#3,$7} END{print "总计有"NR"个账户"}' /etc/passwd
#输出
用户名 UID 解释器
root 0 /bin/bash
bin 1 /sbin/nologin
...
总计有40个账户

#说明
上面的命令在读取/etc/passwd文件内容之前先在屏幕输出标题“用户名UID解释器”，接着逐行读取文件每行内容，每读取一行数据就以冒号进行分隔，打印输出第1列、第3列和第7列，当所有数据行都读取完毕后，在屏幕上打印输出常量与变量，打印常量字符串“总计有”和“个账户”，打印变量NR，而读取完所有行后再打印NR，NR为最后一行的行号。
```

在**awk**中变量不需要定义就可以直接使用，作为字符处理时未定义的变量默认值为空，作为数字处理时未定义的变量默认值为0。使用双引号引用的[和]被识别为常量字符串，输入什么即打印什么。

8.2 条件判断

awk的if判断语句同样支持单分支、双分支及多分支判断。**awk**的单分支if判断语法格式如下，if判断后面如果只有一个动作指令，则花括号{}可以省略，如果if判断后面的指令为多条指令则需要使用花括号{}括起来，多个指令使用分号分隔。

```
## 单分支
if(判断条件) {
动作指令序列;
}

## 双分支
if(判断条件){
动作指令序列1;
}
else {
动作指令序列2;
}

##多分支
if(判断条件){
动作指令序列1;
}
else if(判断条件){
动作指令序列2;
```

```
}  
... ..  
else {  
动作指令序列N;  
}
```

举例：

```
ps -eo user,pid,pcpu,comm | awk '{if($3>0.5){print}}'    #打印第三列大于0.5的数据行
```

```
awk -F: \   
'{if($3<1000){x++}else{y++}} \   
END{print "系统用户个数: "x,"普通用户个数:"y}' /etc/passwd
```

```
awk \   
'{ \   
if($2>=90) {print $1,"\t完美人生"} \   
else if($2>=80) {print $1, "\t优良品质"} \   
else {print $1, "\t其他"} \   
}' test.txt
```

8.3 数组与循环

awk支持关联数组，数组的索引下标可以不是连续的数字，索引下标可以是任意字符或数字，当使用数字作为索引时**awk**会自动将数字转换为字符，如果直接使用字符作索引则需要使用引号括起来。

直接使用数组名加索引下标即可调用数组的值。因为数字索引会被自动转换为字符，所以在定义数组使用数字的情况下，调用数组时也可以使用字符的形式调用。 **数组定义**

```
#一维数组  
数组名[索引]=值  
#多维数组  
数组名[索引1][索引2]=值 或者 数组名[索引1,索引2]=值
```

举例：

```
awk 'BEGIN{a[0]=11;tom["age"]=22; print a[0],a["0"],tom["age"]}'
```

for循环

```
## 方式一：
for(变量 in 数组名) {
    动作指令序列
}

## 方式二：
for(表达式1; 表达式2; 表达式3) {
    动作指令序列
}
```

举例

```
awk 'BEGIN{ \
a[10]=11;a[88]=22;a["book"]=33;a["work"]="home"; \
for (i in a){print i,a[i]} \
}
##通过循环打印索引和数组元素的值
```

注意：这种通过循环的方式输出数组索引或者元素值时并不会按照输入的顺序输出，它是无序的。

```
awk '{ \
for(i=1;i<=NF;i++) \
    {if($i~/apple/) x++} \
}
END{print x}' test.txt
```

#说明：其中一个循环是隐含循环，awk读取一行数据动作指令就会执行一次，test.txt文件包含3行数据，也就是指令（for (i=1; i <=NF; i++) {if (\$i ~/apple/) x++ }）会被重复执行3次，而每执行一次动作指令又会触发for循环被执行，而for循环以NF列数为标准定义循环次数，如果数据行有8列，则for循环中的变量i就会从1循环到8，循环8次的目的就是逐列判断是否包含关键词apple。

while循环

#当while的条件判断为真时则循环执行动作指令，直到条件判断为假时循环结束。

```
while(条件判断){
    动作指令序列;
}
```

例：

```
awk 'BEGIN{i=1;while(i<5){print i;i++}}'
```

awk提供了continue、break和exit循环中断语句，用法与C语法一样。

8.4 awk函数

内置函数

类型	函数	说明
内置IO 函数	getline	让awk立刻读取下一行数据（读取下一条记录并复制给\$0，并重新设置NF、NR和FNR）
内置数 值函数	cos(expr)	返回expr的cosine值
	sin(expr)	返回expr的sine值
	sqrt(expr)	返回expr的平方根
	int (expr)	取整函数，截取整数部分数值
	rand()	返回0到1之间的随机数N(0<=N<1)
	srand ([expr])	使用expr定义的新的随机数种子，没有expr时使用当前系统时间作为随机数种子
内置字 符串函 数	length([s])	统计字符串的长度，如果不指定字符串则统计\$0的长度
	index (字符串1, 字符串2)	返回字符串2在字符串1中的位置
	match(s,r)	根据正则表达式r返回其在字符串s中的坐标位置
	tolower(str)	将字符串转换为小写
	toupper(str)	将字符串转换为大写
	split(字符串，数组，分隔符)	将字符串按照特定的分隔符切片后存储在数组中，如果没有指定分隔符，使用FS定义的分隔符
	gsub(r,s[,t])	将字符串t中所有与正则表达式r匹配的字符串全部替换为s，如果没有指定字符串t，默认对\$0进行替换
	sub(r,s[,t])	与gsub类似，但是仅仅替换第一个匹配的字符串，而不是替换全部

类型	函数	说明
	<code>substr(s,i[,n])</code>	对字符串进行截取，从第i位开始，截取n个字符，如果n没有指定则一直截取到字符串s的末尾位置
内置时间函数	<code>systemtime()</code>	返回当前时间距离1970-01-01 00:00:00有多少秒

注意：awk的起始index是1，而不是0

举例：

df -h的显示：

```
Filesystem Size Used Avail Use Mounted on
/dev/mapper/VolGroup00-LogVol100
          19G  3.6G   15G   21% /
/dev/sda1  99M  14M    81M   15% /boot
tmpfs      141M  0      141M   0%  /dev/shm
```

命令：

```
df -h |awk '{if(NF==1){getline; print$3}; if(NF==6){print $4}}'
```

说明：

1. 读取第1行数据，该行数据包括7个字段，与NF==1和NF==6都不匹配，因此不打印任何数据。
2. 读取第2行数据，该行数据包括1个字段，与NF==1匹配，因此先执行getline，没有执行getline之前\$0的值是/dev/mapper/VolGroup00-LogVol100，执行getline之后，\$0被重新附值为"19G 3.6G 15G 21%/"，此时再打印第3列刚好是磁盘的剩余空间15GB。
3. 读取第3行数据，该行数据包括6个字段，与NF==6匹配，因此直接输出第4列的数值。
4. 读取第4行数据，该行数据包含6个字段，与NF==6匹配，因此直接输出第4列的数值。

```
awk 'BEGIN{srand(99); print rand()}'
```

```
awk 'BEGIN{test="hello the world"; print length(test)}' #打印test的长度
```

```
awk '{print length()}' /etc/shells #返回文件每行的长度
```

```
awk 'BEGIN{test="hello the world"; print index(test, "h")}' #h在变量test的第一个位置，返回1
```

```
awk 'BEGIN{print match("How much?981$", "[0-9]*)}' #数字在第11个位置出现
```

```
awk 'BEGIN{print tolower("THIS IS A TEst")}' #打印 this is a test
```

```
awk 'BEGIN{split("hello the word", test); print test[3],test[2],test[1]}' #打印结果是world the hello
```

```
awk 'BEGIN{split("hello:the:word", test,":"); print test[3],test[1]}' #打印结果是world hello
```

```
awk 'BEGIN{hi="hello world";gsub("o","0",hi); print hi}' #打印结果是 hell0 w0lrd
```

```
awk 'BEGIN{hi="Hello World"; print substr(hi, 2, 3)}' #从第二位截取三个字符，输出是: ell
```

用户自定义函数 语法：

```
function 函数名(参数列表) {命令序列}
```

举例：

```
awk 'function myfun() {print "hello"} BEGIN {myfun()}'      #打印 hello

awk ' \
function max(x,y) \
if(x>7) {print x} \
else {print y} \
BEGIN {max(8,9)}'      #打印 9
```

9 shell常用扩展

Shell脚本支持七种类型的扩展功能：花括号扩展（brace expansion）、波浪号扩展（tilde expansion）、参数与变量替换（parameter and variable expansion）、命令替换（command substitution）、算术扩展（arithmetic expansion）、单词切割（word splitting）和路径替换（pathname expansion）

9.1 花括号扩展

使用花括号对字符串进行扩展。我们可以在一对花括号中包含一组以分号分隔的字符串或者字符串序列组成一个字符串扩展，注意最终输出的结果以空格分隔。使用该扩展时花括号不可以被引号引用（单引号或双引号），在括号的数量必须是偶数个。

字符串序列后面可以跟一个可选的步长整数，该步长的默认值为1或-1。

使用花括号扩展时在花括号前面和后面都可以添加可选的字符串，且花括号扩展支持嵌套。

举例：

```
echo {r,f,j}      #打印r f j
echo {a..f}       #打印a b c d e f
echo {1..9..2}    #打印1 3 5 7 9
echo "{1..9}"     #打印{1..9}, 使用花括号扩展时不能被引号包含

echo t{o,e{a,m}}p #打印top teap temp
```

9.2 波浪号

波浪号在**Shell**脚本中默认代表当前用户的家目录，我们也可以在波浪号后面跟一个有效的账户登录名称，可以返回特定账户的家目录。但是，注意账户必须是系统中的有效账户。

波浪号扩展中使用`~+`表示当前工作目录，`~-`则表示前一个工作目录。

```
echo ~                #显示当前的家目录，root的话，显示/root/
echo ~nanana         #打印/home/nanana

cd /root/
cd /tmp/
echo ~+              #打印/tmp
echo ~-              #打印/root/
```

9.3 变量替换

变量字符可以放到花括号中，这样可以防止需要扩展的变量字符与其他不需要扩展的字符混淆。如果`$`后面是位置变量且多于一个数字，必须使用`{}`，如`1`、`{11}`、`${12}`。

举例：

```
hi = "Go Go"
echo $hi           #打印 Go Go
echo ${hi}         #打印 Go Go
```

如果变量字符串前面使用感叹号（`!`），可以实现对变量的间接引用，而不是返回变量本身的值。感叹号必须放在花括号里面，且仅能实现对变量的一层间接引用。

```
${!var} : 引用间接变量
如：
v1="v2"
v2="hello"
echo ${!v1} #等价与echo $v2, 输出hello
```

变量替换操作还可以测试变量是否存在及是否为空，若变量不存在或为空，则可以为变量设置一个默认值。**Shell**脚本支持多种形式的变量测试与替换功能

语法格式	功能描述
<code>\${变量:-关键字}</code>	如果变量未定义或值为空，则返回关键字，否则返回变量的值
<code>\${变量:=关键字}</code>	如果变量未定义或值为空，则将关键字赋值给变量并返回结果，否则返回变量的值
<code>\${变量:?关键字}</code>	如果变量未定义或值为空，则通过标准错误显示包含关键字的错误信息，否则返回变量的值
<code>\${变量:+关键字}</code>	如果变量未定义或值为空，就直接返回空，否则返回关键字

举例：

```
## 例1
echo $animal      #打印空
echo ${animal:-dog} #打印dog
echo $animal      #打印空，仅仅返回关键字dog，不会因此改变animals的值，所以animals的值还是空。

## 例2
echo $animal      #打印空
echo ${animal:=lion} #打印lion，并给animal赋值
echo $animal      #打印lion

## 例3
echo $input
echo ${input:?}"你没有输入数据"} #-bash: input: 你没有输入数据

## 例4
echo $key          #打印空
echo ${key:+lock}  #打印空
key=heart
echo ${key:+lock}  #打印lock
```

9.4 变量替换

通过（命令）或者`命令`实现命令替换，推荐使用（命令）这种方式，该方式支持嵌套的命令替换。

举例：

```
echo -e "system CPU load:\n $(date +%Y-%m-%d)"
```

9.5 算术替换

通过算术替换扩展可以进行算术计算并返回计算结果，算术替换扩展的格式为 `( ( ) )`，也可以使用 `[]` 的形式，算术扩展支持嵌套。

举例：

```
i=1
echo $((i++))    #打印1
echo $i          #打印2
```

9.6 进程替换

命令替换将一个命令的输出结果返回并且赋值给变量，而进程替换则将进程的返回结果通过命名管道的方式传递给另一个进程。

进程替换的语法格式为：`< (命令) 或者 > (命令)`。一旦使用了进程替换功能，系统将会在 `/dev/fd/` 目录下创建文件描述符文件，通过该文件描述符将进程的输出结果传递给其他进程。

```
who | wc -l          #打印5
wc -l <(who)         #打印 5 /dev/fd/63
```

`< (who)` 会将 `who` 命令产生的结果保存到 `/dev/fd/63` 这个文件描述符中，并将该文件描述符作为 `wc -l` 命令的输入参数，最终 `wc -l < (who)` 的输出结果说明了在 `/dev/fd/63` 这个文件中包含 5 行内容。需要注意的是，文件描述符是实时动态生成的，所以当进程执行完毕，再使用 `ls` 查看该文件描述符时会提示没有该文件。

9.7 单词切割

单词切割也叫分词，**Shell** 使用 **IFS** 变量进行分词处理，默认使用 **IFS** 变量的值作为分隔符，对输入数据进行分词处理后再执行命令。如果没有自定义 **IFS**，则默认值为空格、**Tab** 制表符和换行符。

```
read -p "请输入3个字符：" x y z          ##如果输入1 2 3, 则x=1, y=2,z=3;如果输入
123, 则x=123, y和z为空
```

9.8 路径替换

路径或文件名，可使用**basename**和**dirname**这两个外部命令，还可以截取一个路径中的路径部分或者文件名部分的内容。

```
basename /a/b/c/d.txt      #返回d.txt
dirname /a/b/c/d.txt       #返回/a/b/c
```

常用命令

WC

- **-c**: 输出数据的字节数
- **-l**: 输出数据的行数
- **-w**: 输出数据的单词数